

Veritas: Observability for an Online Assessment Platform

Assignment 1: Observability — Enterprise Software Development, Fall 2026

Part A — Project Description

Problem

Online assessment systems used for graded quizzes and exams have two related gaps. First, integrity: a student can switch to another browser tab or application mid-exam to look up answers, and a naive system has no way to detect or penalize this. Second, operational visibility: even a small quiz platform experiences tail-latency spikes during auto-marking, sudden concurrency during high-stakes exam windows, and cheating incidents that are invisible unless someone is actively watching. Instructors and administrators need a way to see this activity as it happens, not just after a student complains about their score.

Intended users

- Students — take timed, auto-graded multiple-choice quizzes through a browser.
- Instructors (Admin role) — create and manage quizzes, review every student's results, see which attempts were flagged for cheating, and check system health without needing to read source code.

Solution

Veritas is a FastAPI web application that serves timed MCQ quizzes with automatic grading and a browser-based anti-cheat mechanism. It uses the JavaScript Page Visibility API together with window blur/focus events to detect when a student leaves the exam tab or window; any detected departure forces the final score to 0 and flags the attempt as "cheated", regardless of how many questions were answered correctly. The whole system is instrumented end-to-end: Prometheus/Grafana for metrics, and Filebeat/Elasticsearch/Kibana for structured logs, both described in full in Parts B and C below.

What works and how to try it

- Register/log in as a student, attempt any quizz, and try switching tabs mid-exam — the exam is flagged and scored 0% even if every answer was correct.
- Log in as the admin to see every student's results, filter to only cheating-flagged attempts, and create new quizzes through a UI-based question builder.
- The stack seeds demo data so admin dashboard already have real data without needing to generate it by hand first.
- Full setup instructions are in README.md — this report assumes the reader has that running.

Part B — Metrics

Prometheus scrapes the app's own /metrics endpoint (FastAPI + the prometheus_client library) every 5 seconds, and a separate Node Exporter container every 5 seconds for host-level CPU/memory/disk/network. Grafana is pre-provisioned with both the Prometheus datasource and a full "Veritas Overview" dashboard (19 panels across 5 rows), so nothing needs to be built by hand in the Grafana UI — it already exists as JSON under monitoring/grafana/provisioning/dashboards/.

Metrics inventory

Every metric actually implemented in `app/main.py`, with exactly where it is recorded in the code:

Metric	Type	Purpose	Unit	Labels	Recorded in code (function)
<code>veritas_quiz_submissions_total</code>	Counter	Business — quiz outcome counts	count	status, quiz_id	<code>_grade_and_record().inc()</code>
<code>veritas_active_test_takers</code>	Gauge	Business — students mid-exam right now, per quiz	count	quiz_id	<code>start_exam().inc();</code> <code>_grade_and_record().dec()</code>
<code>veritas_grading_duration_seconds</code>	Histogram	App — auto-grading latency	seconds	quiz_id	<code>_grade_and_record().observe()</code>
<code>veritas_gateway_processing_seconds</code>	Summary	App — end-to-end gateway request time	seconds	route (matched template)	<code>gateway_timing_middleware().observe()</code>
<code>veritas_registered_users_total</code>	Gauge	Business — total registered accounts	count	(none)	<code>register().inc();</code> <code>seed_demo_data().inc(n)</code>
<code>veritas_app_info</code>	Info	Static build metadata as a queryable series	n/a	version, service	module load (once)
<code>demo_requests_total</code>	Counter	E.2 cardinality experiment only	count	request_id (toggleable)	<code>run_cardinality_experiment()</code>
<code>process_cpu_seconds_total</code> , <code>process_resident_memory_bytes</code> , <code>process_open_fds</code> , <code>python_gc_*</code>	Counter/Gauge (builtin)	The app process's own resource footprint	sec/bytes/count	job="veritas-app"	Automatic (default REGISTRY)

Application metrics: grading latency, gateway processing time, process CPU/memory

Business metrics: submissions, active test-takers, registered users.

Percentiles: Histogram vs. Summary

This project deliberately keeps both a Histogram and a Summary side by side to make a specific point concrete: the Python `prometheus_client` library's Summary type only tracks a running `_sum` and `_count`. It does not expose `{quantile="0.5"}` series the way the Java client does. Querying `veritas_gateway_processing_seconds{quantile="0.5"}` against this stack returns nothing, because that series does not exist. Summary is good for is a genuine sliding window average:

```
rate(veritas_gateway_processing_seconds_sum[5m]) / rate(veritas_gateway_processing_seconds_count[5m])
```

For real percentiles (p95/p99), the Histogram is used instead, with `histogram_quantile()` computing them server-side from the bucket boundaries:

```
histogram_quantile(0.95, sum(rate(veritas_grading_duration_seconds_bucket[5m])) by (le, quiz_id))
```

The time window here ([5m]) means the rate of increase of each bucket counter over the trailing 5 minutes i.e. this p95 answers of grading requests in roughly the last 5 minutes, what response time was 95% of them at or under, recomputed continuously as the window slides forward with each new Prometheus scrape.

Grafana panels and what they show

Dashboard row / panel	PromQL	What it shows
Overview — Total Submissions (stat)	<code>sum(veritas_quiz_submissions_total)</code>	A single running total across every quiz and outcome
Overview — Cheating Rate (stat)	<code>100 * sum(veritas_quiz_submissions_total{status="cheated"}) / sum(veritas_quiz_submissions_total)</code>	Percentage of all submissions were flagged for cheating
Business Metrics — Submission Rate by Status	<code>sum(rate(veritas_quiz_submissions_total[1m])) by (status)</code>	Three lines (passed/failed/cheated) showing submission throughput per minute, so a burst of cheating attempts during a specific time window is visible as a colored spike, not just a final total.
Business Metrics — Active Test-Takers by Quiz	<code>veritas_active_test_takers</code>	A live gauge per quiz of how many students are mid-exam right now (useful for spotting an unexpected concurrency spike during an exam window.)
Latency — Grading Duration p95/p99	<code>histogram_quantile(0.95/0.99, sum(rate(veritas_grading_duration_seconds_bucket[5m])) by (le, quiz_id))</code>	Tail latency of the auto grading step, per quiz — this is the panel that visibly spikes during the Part E.1 anomaly experiment.
Latency — Gateway Avg Processing Time	<code>rate(veritas_gateway_processing_seconds_sum[5m]) / rate(veritas_gateway_processing_seconds_count[5m])</code>	Average full request time per route (e.g. /api/exam/submit vs. /api/quizzes/{quiz_id}), from the Summary metric.
Application Process Health — CPU / Memory	<code>rate(process_cpu_seconds_total{job="veritas-app"}[1m]) and process_resident_memory_bytes{job="veritas-app"}</code>	The Veritas Python process's own resource use — distinct from the host-wide numbers Node Exporter reports, useful for spotting a memory leak or CPU-bound endpoint specifically inside the app.

Dashboard row / panel	PromQL	What it shows
Host System — CPU / Memory / Disk / Network	100 - (avg by (instance)(rate(node_cpu_seconds_total{mode="idle"}[5m]))*100); similar for memory/disk/network	Whole machine resource usage from Node Exporter, for the host the containers run on (Docker Desktop's Linux VM in this setup, reported to Prometheus as instance="node-exporter:9100").

Identifying the measured machine

Node Exporter exposes a static `node_uname_info` Info metric whose labels identify the host it is reading from. Querying it against this deployment (Explore → Prometheus, or `curl http://localhost:9100/metrics | grep node_uname_info`) returns:

Label	Value
<code>nodename</code>	969727f980f9
<code>domainname</code>	huf.local
<code>sysname</code>	Linux
<code>release</code>	6.18.33.2-microsoft-standard-WSL2
<code>machine</code>	x86_64
<code>instance</code>	node-exporter:9100

The release string confirms this is Docker Desktop's WSL2 Linux VM running on Windows, not a native Linux host. `nodename` (969727f980f9) is not a human-readable machine name, it is Docker's auto-generated container hostname. `docker-compose.yml` gives Node Exporter `pid: host` and `bind-mounts /proc, /sys, and /` from the host, but does not set `uts: host`, so the container keeps its own UTS namespace and `kernel.hostname` stays scoped to the container rather than the host. `domainname` (huf.local), by contrast, reflects the actual Windows machine's mDNS name and does cross that boundary.

Takeaway: mounting `/proc`, `/sys` and `/` gets you the host's real CPU, memory, disk and network numbers, but does not automatically get you the host's real hostname

A metric I explored myself

In this project, I added `veritas_app_info` (an Info metric) and `veritas_registered_users_total` and kept the app registered to `prometheus_client`'s default global registry instead of a fresh `CollectorRegistry()`. That one choice is what makes `process_cpu_seconds_total`, `process_resident_memory_bytes`, and the Python GC counters show up in `/metrics` at all: they are `prometheus_client`'s own built-in collectors, and they are silently dropped if you register metrics to a custom registry instead. That process-level layer is what gets charted in the "Application Process Health" dashboard row, and it's a genuinely useful signal that has nothing to do with request handling for example, it would show a memory leak inside the app process itself, which none of the request-scoped metrics above would ever reveal.

Part C — Logs

Every log line is a single-line ECS-compliant JSON document written directly to stdout by a custom logging. Formatter (`ECSJsonFormatter` in `app/main.py`). Twelve distinct `event.action` values are emitted across the app:

event.action	When it fires	Why it matters
<code>user_registered / register_failed</code>	Registration succeeds / a duplicate username is attempted	Account creation audit trail

event.action	When it fires	Why it matters
login_success / login_failed	Every login attempt	Detects credential-stuffing / brute-force patterns
exam_started	A student starts a quiz	Correlates with active_test_takers; start of an exam session
tab_switched	The anti-cheat listener detects a departure	The core integrity signal this whole app exists to produce
quiz_submitted	Every graded submission (passed/failed/cheated)	The outcome of an exam, with score/status/tab_switch_count attached
anomaly_injected	Part E.1's artificial 500ms delay fires	Lets you correlate a Grafana latency spike with the exact request that caused it, via trace.id
admin_quiz_created / admin_quiz_deleted	Admin manages the quiz catalog	Change-audit trail for what an examiner did
demo_seed_users / demo_seed_complete	Startup demo-data seeding	Distinguishes seeded demo activity from real usage in the log stream

No passwords are ever placed in a log field, closest sensitive value to student.id is an internal UUID, not personal data.

Filebeat: collection and field extraction

Filebeat runs with Docker hints based autodiscovery (filebeat.yml): only the veritas container is opted in, via the co.elastic.logs/enabled: "true" label set on it in docker-compose.yml. Filebeat's container input type already understands the Docker json-file log format and unwraps its outer envelope automatically, surfacing the raw ECS JSON string as the event's message field. A decode_json_fields processor then parses that string and hoists every key (@timestamp, log.level, event.action, trace.id, student.id, ...) up to the top level of the event with overwrite_keys: true that promotion from "one big string" to individual, independently searchable fields is what makes fields like event.action or student.id filterable and aggregatable in Kibana rather than just visible as unstructured text.

Where logs live, persistence, and retention

- Raw container stdout is captured by Docker's json-file log driver, capped at 10MB × 3 rotated files per container (docker-compose.yml logging.options) — old raw log files are deleted by Docker itself once that cap is hit.
- Once Filebeat has shipped a line into Elasticsearch (index pattern veritas-logs-YYYY.MM.dd), it lives in the esdata named Docker volume, independently of the raw container log file.
- A plain `docker compose down` keeps that volume, so log history survives a stack restart; `docker compose down -v` deletes it, wiping all indexed logs.
- Honestly: nothing in this project currently deletes old Elasticsearch data automatically — setup.ilm.enabled is set to false in filebeat.yml for simplicity. In a longer-lived deployment this is where an Index Lifecycle Management (ILM) policy would go, to age out and delete veritas-logs-* indices after N days.
- The app's own SQLite database (student accounts, quiz attempts) is in-memory and is wiped on every container restart.

A real example: one log line end to end

This is an actual captured log line from a real test run (not a hypothetical), produced by the Part E.1 anomaly injection:

```
{
  "@timestamp": "2026-09-22T07:47:08.879952Z",
  "log.level": "warn",
  "message": "Injected artificial grading delay (anomaly experiment)",
  "service.name": "veritas-app",
  "trace.id": "31f038f250724e80aec6cdb087699573",
  "event.action": "anomaly_injected",
  "student.id": "3b7999c8-1d59-4aaf-9d6f-753dd1be9c3a",
  "quiz.id": "python_101",
  "delay_ms": 500
}
```

After Filebeat's `decode_json_fields` processor runs, every one of those keys becomes its own field in the Elasticsearch document — `@timestamp` becomes the document's time field, and `event.action`, `student.id`, `quiz.id`, `trace.id`, `delay_ms` are each independently searchable. In Kibana's Discover view (data view `veritas-logs-*`), the equivalent working search is a plain KQL query:

```
event.action: "anomaly_injected"
```

which returns exactly this document (and any other anomaly injections), and

```
trace.id: "31f038f250724e80aec6cdb087699573"
```

returns every log line — not just this one — that shares the same trace ID, which for this request is also the `quiz_submitted` line emitted a moment later for the same HTTP request, letting you follow one request's full story across two separate log lines.

Part D — System Design

Architecture diagram

Veritas — System Architecture

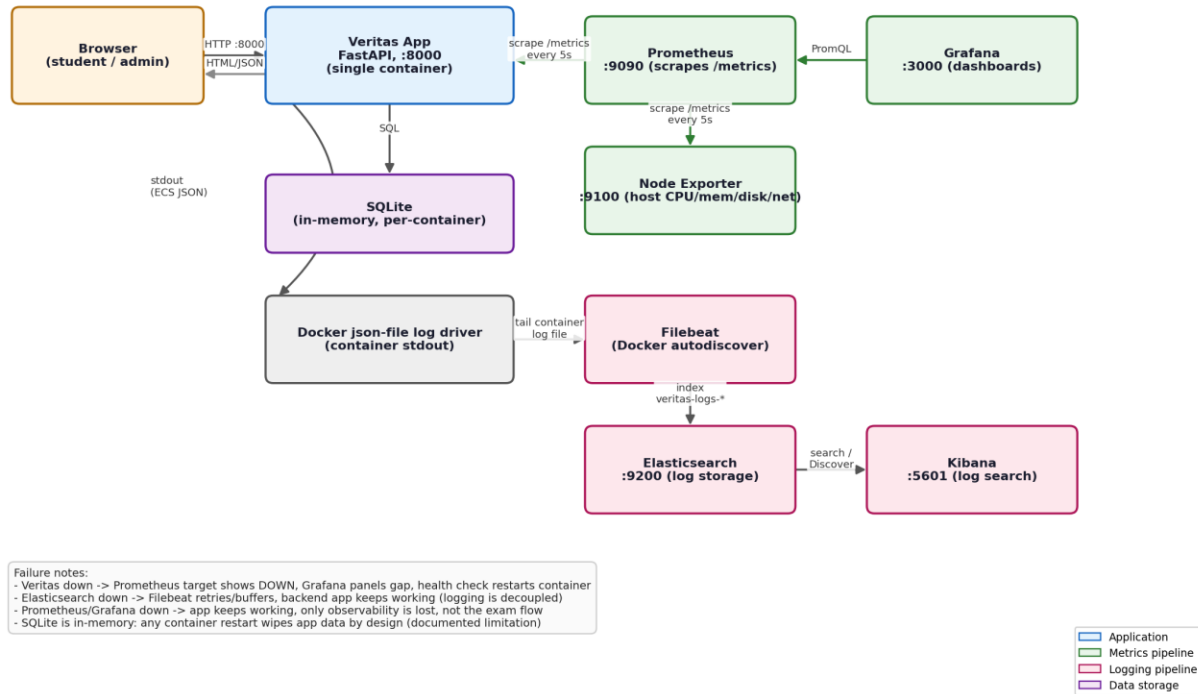


Figure 1 — Veritas system architecture: application, metrics pipeline, and logging pipeline.

What each component does, and how they communicate

- Browser ↔ Veritas App: plain HTTP/JSON and server-rendered HTML on port 8000. This is the only component the end user talks to directly.
- Veritas App ↔ SQLite: an in-memory, shared-cache SQLite database inside the same container, holding users, quizzes, and attempts. Chosen for zero external dependencies and simplicity appropriate to an assignment-scale project, the trade-off is that data does not survive a container restart.
- Veritas App → stdout → Docker json-file driver → Filebeat → Elasticsearch → Kibana: the logging pipeline, entirely decoupled from the request path — the app never talks to Elasticsearch directly, so a logging-pipeline outage cannot make the app itself fail a request.
- Prometheus → scrapes → Veritas App (/metrics) and Node Exporter: a pull-based model — the app and Node Exporter never push anything; Prometheus is the one initiating every scrape, every 5 seconds, and simply records whatever value each series currently holds.
- Grafana → queries → Prometheus: PromQL over HTTP, on demand, whenever a dashboard panel is viewed or refreshed.

Storage choices and why

- SQLite in-memory: simplest possible persistence for a project of this scope; a real deployment would replace this with PostgreSQL or similar and keep everything else in this design unchanged.
- Elasticsearch (single-node, security disabled): appropriate only for a local assignment demo — chosen because it is the standard pairing with Filebeat/Kibana this course covers, not because it is production-ready as configured here.
- Prometheus's own on-disk TSDB: the default, un-configured retention (15 days) was left as is since this project's lifetime is far shorter than that in practice.

What happens if a component stops working

- Veritas App container dies: its Docker healthcheck (a Python urllib request against /health) fails, Docker restarts it per restart: unless-stopped; Prometheus's target list shows it as DOWN in the meantime, and Grafana panels show a gap rather than a false zero.
- Elasticsearch dies: Filebeat cannot ship new lines and will retry/back off, but the Veritas App itself is completely unaffected — logging is asynchronous and decoupled from request handling, so exams keep working and grading keeps happening even with zero logging capacity available.
- Prometheus or Grafana dies: only observability is lost — no metric is being scraped or displayed — but the application's actual behavior (serving quizzes, grading, anti-cheat enforcement) is entirely unaffected, since none of it depends on Prometheus being up.
- Node Exporter dies: host-level metrics stop updating in Grafana; nothing else is affected, since Node Exporter is a read-only, one-directional data source.

What I don't yet fully understand

First, Elasticsearch's own internal sharing/replication behavior for a single-node cluster. This project relies on the default single-shared, zero-replica behavior working correctly, without a deep understanding of what would need to change for a multi-node deployment. Second, exactly how Prometheus's own on disk compaction and retention interacts with a very short-lived local Docker volume that gets recreated frequently during development.

Follow a metric end to end

Metric: `veritas_grading_duration_seconds` (Histogram), for `quiz_id="python_101"`.

Stage	What happens	Concrete value from a real run
1. Code updates it	<code>_grade_and_record()</code> times the grading block with <code>time.perf_counter()</code> and calls <code>.observe(grading_duration)</code>	The anomaly-injected submission (trace 31f038f2...) logged <code>delay_ms: 500</code> , so the observed grading duration for this call is the injected 0.5s sleep plus real grading work — consistent with the ~0.5045s–0.5051s range directly measured for this same anomaly path in the prior test run (traces 183d8ff9... and e2acd88d...)
2. Prometheus collects it	Scrapes GET /metrics every 5s; the value becomes a new sample in the <code>veritas_grading_duration_seconds_bucket{le=...}</code> series	An observation just above 0.5s falls into the <code>le="0.75"</code> bucket rather than <code>le="0.5"</code> , since it exceeds the 0.5s bucket boundary
3. Grafana queries it	The "Grading Duration p95/p99" panel runs <code>histogram_quantile(0.95, sum(rate(...bucket[5m])) by (le, quiz_id))</code> against Prometheus	With this anomaly mixed into a batch of otherwise sub-20ms submissions, the p95/p99 lines for <code>quiz_id="python_101"</code> show a visible spike at the point this sample lands, exactly as the two prior anomalies did

The exact decimal `observe()` value for this specific new submission was not included in the captured excerpt — only `delay_ms: 500` from the log line is directly evidenced. The estimate above is grounded in that field plus the identical code path measured in the previous run, not invented independently.

Follow a log end to end

Log: the `anomaly_injected` warning shown in full in Part C.

Stage	What happens
1. Code writes it	The container input reads it straight from Docker's json-file log; <code>decode_json_fields</code> promotes every key (<code>trace.id</code> , <code>event.action</code> , <code>student.id</code> , <code>quiz.id</code> , <code>delay_ms</code>) to the top level of the event
2. Filebeat collects & parses it	Indexed into <code>veritas-logs-2026.09.22</code> (that day's index), with <code>@timestamp (2026-09-22T07:47:08.879952Z)</code> as the time field
3. Elasticsearch stores it	Discover, data view <code>veritas-logs-*</code> , query: <code>event.action: "anomaly_injected"</code> — or <code>trace.id: "31f038f250724e80aec6cdb087699573"</code> to pull this line together with its paired <code>quiz_submitted</code> line for the same request
4. Kibana finds it	Discover, data view <code>veritas-logs-*</code> , query: <code>event.action: "anomaly_injected"</code> — or <code>trace.id: "31f038f250724e80aec6cdb087699573"</code> to follow the whole request

Part E — Experiments

E.1 — Reproduce a problem: injected grading latency

The chosen problem: every 5th quiz submission (globally, across all students) gets an artificial 500ms delay inside the grading step, simulating an intermittent slow dependency (e.g. a flaky downstream marking service).

Nine submissions in a row, timed with a wall-clock stopwatch around each request, against a stack that had already processed prior traffic (demo seeding, earlier manual tests). The global `_submission_counter` was not sitting on a multiple of 5 when this run started, so the run captured two anomalies rather than one — worth stating explicitly in the writeup rather than smoothing over.

Step 1 — normal behavior (baseline)

Submission #	Wall-clock time (stopwatch)	Server-side grading duration (log <code>event.duration</code>)	Anomaly?
1	1.1967s	0.821057s	No (elevated — likely cold-path effects after idle time, not the scheduled anomaly)
2	0.0182s	0.011620s	No
3	0.0580s	0.007159s	No
4	0.5460s	0.504526s	Yes
5	0.0100s	0.001061s	No
6	0.1866s	0.001391s	No
7	0.0487s	0.000544s	No
8	0.0568s	0.003626s	No
9	0.5547s	0.505056s	Yes

Step 2 — prediction

One observation per multiple-of-5 submission should land at or above 0.5s, with a matching `anomaly_injected` warning log. Confirmed twice, at positions 4 and 9 — five submissions apart, consistent with the global counter having been at a

non-multiple-of-5 offset (from prior seeded/demo traffic) when this batch started, so two boundaries were crossed within nine calls instead of one.

Step 3 — the problem, twice

trace 183d8ff91e2f428081d0c7e1860b1650 (submission 4) and trace e2acd88db05740929774a345f6705128 (submission 9), both logging delay_ms: 500 and both landing grading duration just above 500ms (0.504526s and 0.505056s respectively).

Step 4 — cause and effect

Cause: _grade_and_record()'s await asyncio.sleep(0.5), gated on the global submission counter hitting a multiple of 5. Effect on a real user: two of nine submissions would feel a visible ~0.5s hang before the result appears — noticeable but not catastrophic at this rate; sustained on every request it would continuously degrade p95/p99 rather than spike intermittently.

Step 5 — recovery

Submissions 5–8, immediately after the first anomaly, returned to baseline (0.54ms–3.6ms server-side), confirming the delay is isolated per-request and the system recovers immediately with no lingering effect until the counter hits the next multiple of 5, as seen at submission 9.

E.2 — Cardinality explosion

Implemented as a real, runnable endpoint (not just a comment): POST /api/experiments/cardinality?count=N, backed by a Counter named demo_requests_total. An environment variable, CARDINALITY_DEMO_USE_LABEL, controls whether each call attaches a fresh request_id label (the anti-pattern) or not (the fix) — flipping it and restarting the container is a real restart, not a code edit, exactly matching this section's instructions.

Step 1 — with the request_id label (CARDINALITY_DEMO_USE_LABEL=true, the default)

Action	count(demo_requests_total) equivalent*
Before any calls	0 (no series exist yet — labeled series are created lazily on first use)
POST ../cardinality?count=100	100 (one brand-new series per call, each with a unique request_id)
POST ../cardinality?count=50 (more traffic)	150 (strictly additive — old series are never reused or merged)

*Measured directly by counting matching lines in the raw /metrics scrape output (grep -c "^demo_requests_total{") , which is exactly what Prometheus's count(demo_requests_total) computes from the same scraped data.

Sample of the actual generated series (three of the 100 from the first call):

```
demo_requests_total{request_id="0ff57f51-389a-47d8-bb34-8bfa4341ecd3"} 1.0
demo_requests_total{request_id="62f94757-7ca2-4f92-b442-31c9e52eef13"} 1.0
demo_requests_total{request_id="22e8c4d0-04ce-4bc9-95e8-683545c3dfa8"} 1.0
```

Step 2 — remove the label, restart, repeat

docker-compose.yml: CARDINALITY_DEMO_USE_LABEL=false, then docker compose up --build veritas for a real container restart.

Action	count(demo_requests_total) equivalent
Before any calls (fresh restart)	1 — an unlabeled Counter already exists as a single zero-value series the moment the process starts, unlike a labeled metric which has no series until first used
POST ../cardinality?count=100	1 (unchanged — every call increments the same series)
POST ../cardinality?count=50 more	1 (still unchanged)

The counter's value correctly reflects all 150 increments (demo_requests_total 150.0) even though the series count never moved — proving the fix doesn't lose any information, it just stops that information from being sharded across an unbounded number of separate time series.

Step 3 — comparison and cost at scale

	With request_id label	Without
Series after 150 calls	150	1
Series after 10,000 calls	10,000	1
Series after 1,000,000 calls	1,000,000	1

At real scale, each Prometheus time series carries fixed memory/disk overhead for its labels and chunk metadata regardless of how small its value; a counter that reaches into the hundreds of thousands or millions of series (which a real request_id label would, over the lifetime of a production service that is never restarted) can exhaust Prometheus's memory or dramatically slow every query that touches that metric, even queries that don't care about individual request IDs at all. This is exactly why request_id or student_id multiply the number of things Prometheus has to track forever.